

FOCI projekt - fejlesztői átadási dokumentum

Állapot: 2026-03-29 | Cél: a projekt zavartalan folytatása új fejlesztővel vagy új chatben

Projekt fókusz	Baráti foci / amatőr csapatesemény-szervezés backend + egyszerű frontend demo
Technológia	Node.js, Express, PostgreSQL, JWT, bcrypt, dotenv, HTML/CSS/JS demo UI
Kész állapot	Team, event, jelentkezés, várólista, lemondás, automatikus elreléptetés, auth login, lista- és részletezés
Kritikus következő lépés	Auth teljes végigvezetése: createTeam és createEvent áttérése req.user.id használatára; userId eltávolítás

1. Projektcél és jelenlegi termékirány

A projekt célja egy amatőr fociszervező rendszer felépítése, amely csapatokat, tagokat, eseményeket és eseményjelentkezéseket kezel. A stratégiai irány a baráti foci szervezésből indul, de a modell később tornaszervezési modul felé bővíthető.

A jelenlegi rendszer fókusza nem chat és nem fizetés, hanem a strukturált eseményszervezés: ki jön, ki várólistás, ki mondta le, és ha valaki kiesik, ki lép automatikusan a helyére.

A legfontosabb üzleti mag már működik: várólista + automatikus elreléptetés. Ez a projekt legerősebb része, ezt kell megőrizni és tovább csiszolni.

2. Elkészült funkciók

Csapat létrehozása (POST /api/teams).

Csapat lekérdezése taglistával (GET /api/teams/:teamId).

Esemény létrehozása csapathoz kötve (POST /api/teams/:teamId/events).

Esemény részletes lekérdezése (GET /api/events/:eventId).

Csapat összes eseményének listázása aggregált számlálókkal (GET /api/teams/:teamId/events).

Jelentkezés eseményre (POST /api/events/:eventId/register).

Lemondás eseményről (POST /api/events/:eventId/cancel).

Várólistás felhasználó automatikus elreléptetése, ha egy 'going' státuszú játékos lemond.

JWT login (POST /api/auth/login) és saját profil lekérdezése (GET /api/auth/me).

Egyszerű frontend demo oldal, amely listázza a csapat eseményeit, megjeleníti az esemény részleteit, és register/cancel gombokkal meghívja az API-kat.

3. Architektúra

Backend: Express alapú REST API.

Adatbázis: PostgreSQL. A business logic a Node oldalon van, SQL-lel támogatva.

Auth: JWT Bearer token, 'requireAuth' middleware.

Frontend demo: külön HTML alapú egyszerű kliens, amely a localhost:3000 API-t hívja.

A projekt jelenleg fejlesztői környezetben fut, nem deployolt éles rendszer.

4. Fájlstruktúra és szerepek

```
src/  
  config/
```

```

db.js
controllers/
  authController.js
  teamController.js
  eventController.js
middleware/
  requireAuth.js
routes/
  authRoutes.js
  teamRoutes.js
  eventRoutes.js
tools/
  hashPassword.js
index.js

```

frontend demo:

canvasban létrehozott HTML oldal (eseménylista + részletezés + register/cancel)

5. Adatmodell - fogalmi áttekintés

users - felhasználók, email, státusz, jelszó hash.

teams - csapatok.

team_members - user és team kapcsolata, szerepkörrel és státusszal.

events - esemény alapadatok, min/max players, időpont, helyszín.

event_settings - pályaméret, minőség, borítás, játékidő, ár, szabályok, csereparaméterek.

event_registrations - eseményre jelentkezések, státusz: `going`, `waiting\_list`, `cancelled`, valamint `registered\_at`, `cancelled\_at`, `promoted\_at` mezők.

6. Kulcsfontosságú üzleti logika

Az esemény maximális létszáma már nem kézzel megadott buta szám, hanem számolt érték: `maxPlayers = playersOnFieldTotal + substitutesCount`.

A csere lehet külön engedélyezve vagy letiltva: `substitutesEnabled` + `substitutesCount`. Ha nincs csere, a csere darabszáma 0.

Jelentkezéskor a backend eldönti, hogy a user `going` vagy `waiting\_list` státuszba kerül.

Lemondáskor, ha a felhasználó `going` volt, akkor a legkorábban várólistára került személy automatikusan `going` lesz.

A lekérdezés endpointok frontend-ready aggregált summary-t adnak vissza: `goingCount`, `waitingCount`, `cancelledCount`, `spotsLeft`.

7. Működő endpointok (jelenlegi állapot)

Method	Endpoint	Állapot	Megjegyzés
GET	/api/health	kész	backend + DB elérhetőség teszt
POST	/api/auth/login	kész	JWT token ad vissza
GET	/api/auth/me	kész	Bearer token szükséges
POST	/api/teams	működik, de javítandó	body-ból vár user azonosítót
GET	/api/teams/:teamId	kész	team + members
POST	/api/teams/:teamId/events	működik, de javítandó	body-ból vár createdByUserId-t

Method	Endpoint	Állapot	Megjegyzés
GET	/api/teams/:teamId/events	kész	eseménylista aggregált számlálókkal
GET	/api/events/:eventId	kész	esemény részletei + registration summary
POST	/api/events/:eventId/register	kész	auth védett, req.user.id használat
POST	/api/events/:eventId/cancel	kész	auth védett, auto-promotion logika

8. Jelenlegi ismert technikai hiányosságok és refaktor feladatok

Legfontosabb: a `createTeam` és a `createEvent` még nem teljesen auth-normalizált. Mindkettőben a user azonosító a request body-ból jön, ezt át kell állítani `req.user.id` használatára.

A `POST /api/teams` route jelenleg nincs auth middleware mögé téve. Ezt be kell védeni, és a `createdByUserId` mezőt ki kell venni a kliensből.

A `POST /api/teams/:teamId/events` endpointban a `createdByUserId` még a body része. Ezt ki kell venni, és a validációs üzenetet is ehhez kell igazítani.

Nincs még egységes permission helper réteg (pl. `isTeamMember`, `isCaptain`). A role ellenőrzés az event controllerben kézzel történik.

A `registerForEvent` logika működik, de még nem race-condition biztos. Később sorzár vagy szigorúbb tranzakciós modell kell.

A `teamController.js` feltöltött állapota alapján kétszeres `module.exports` blokk volt, ezt tisztítani kell, hogy a fájl strukturált és egyértelmű legyen.

Nincs még `/api/events/:eventId/me` endpoint, ami megmondaná a frontendnek, hogy a bejelentkezett user milyen státuszban van az adott eseményen.

Nincs még `/api/me/events` endpoint, ami a user saját közelgő/meglévő jelentkezéseit adná vissza.

9. Pontosan mi működött leellenőrizve

Mini tesztesemény létrehozása megtörtént kis max létszámmal.

Peter és Jozsi `going` státuszba kerültek.

Béla `waiting\_list` státuszba került.

Peter lemondása után Béla automatikusan `going` lett.

Ezt DB-beverben ellenőriztük, és a `GET /api/events/:eventId` endpoint válasza is visszaadta helyesen a változásokat.

Auth login működik, `POST /api/auth/login` sikeresen visszaad tokent, és a `GET /api/auth/me` teszt is sikeres volt.

10. Fejlesztési filozófia / irány

Nem újabb random endpointok gyártása a cél, hanem a rendszer lezárása a helyes sorrendben.

A helyes következő irány: auth normalizálás -> permission layer -> user-specifikus endpointok -> majd csak ezután további frontend finomítás.

A backend logika már erős; a legnagyobb kockázat most nem a feature-hiány, hanem az inkonzisztens auth és jogosultságkezelés.

A projekt stratégiai célja továbbra is a baráti foci modulból induló, később tornaszervezésre és döntéstámogatásra bővíthető rendszer.

11. Azonnali következő teendők (prioritási sorrend)

Prioritás	Feladat	Miért fontos
1	createTeam auth-normalizálás	A user ne mondhatta meg bodyből, ki hozza létre a csapatot
2	createEvent auth-normalizálás	Ugyanez az eseménylétrehozásnál
3	teamRoutes auth-védelem	A csapatlétrehozás se legyen nyitott endpoint
4	permission helper réteg	Ne kézzel legyen szétszórva a role-check
5	/api/events/:eventId/me	Frontendnek tudnia kell, hogy a user megy / vár / nincs bent
6	/api/me/events	Saját eseménylista, valódi felhasználói nézet
7	register race-condition védelem	Két egyidejű kattintás ne töltse túl az eseményt

12. Konkrét refaktor célkép

``POST /api/teams`` - auth kötelező, controllerben ``const createdByUserId = req.user.id;``, bodyből a ``createdByUserId`` törlendő.

``POST /api/teams/:teamId/events`` - auth kötelező, controllerben ``const createdByUserId = req.user.id;``, bodyből a ``createdByUserId`` törlendő.

``POST /api/events/:eventId/register`` - már auth-alapú, ezt meg kell tartani.

``POST /api/events/:eventId/cancel`` - már auth-alapú, ezt meg kell tartani.

A frontendben el kell tüntetni minden kézi ``userId`` küldésnek, kivéve ideiglenes debug mezők, amiket később ki kell venni.

13. Frontend demo állapot

Készült egy egyszerű HTML alapú demo oldal, amely csapat eseményeit listázza, betölti a kiválasztott esemény részleteit, és gombokkal hívja a ``register`` / ``cancel`` endpointokat.

A jelenlegi frontend még fejlesztési/demo állapotú. A login token kezelés nincs teljesen rávezetve a végleges UX-re.

A frontend jelenleg főleg az API validálására szolgál, nem végleges UI.

14. Környezeti és futtatási információk

Fejlesztési környezet Windows gépen, CMD használatával. PowerShell több ponton problémát okozott (``npm.ps1`` policy), ezért a stabil workflow: `CMD + `npm.cmd run dev``.

Futtatás: ``npm.cmd run dev``.

Backend health ellenőrzés: ``GET /api/health``.

Fontos: a `.env` fájlban DB és JWT konfiguráció van. Ezeket új fejlesztésnek át lehet adni, de publikus dokumentumba nem ajánlott teljes titkokkal kitenni.

Package-ek között már bent vannak az authhoz szükséges csomagok: ``bcryptjs``, ``jsonwebtoken``, ``express``, ``pg``, ``dotenv``, ``cors``, ``nodemon``.

15. Mit kell bemásolni egy új chatbe a zökkenőmentes folytatáshoz

Egy Node.js + Express + PostgreSQL alapú foci esemény backend projekten dolgozunk. A rendszer jelenlegi állapota:

- JWT auth kész és működik

- team, event, register, cancel, waiting list logika kész
- GET /api/events/:eventId és GET /api/teams/:teamId/events működik
- frontend demo oldal elkészült egyszer HTML-ben
- a kritikus következő lépés az auth teljes végigvezetése

Kérlek innen folytassuk:

1. createTeam auth-normalizálás
2. createEvent auth-normalizálás
3. permission middleware / helper réteg
4. /api/events/:eventId/me
5. /api/me/events

Fontos: a register és cancel már req.user.id alapú. A createTeam és createEvent még javítandó.

16. Záró értékelés

A projekt jelenlegi állapota kifejezetten erős MVP backend. A legnehezebb üzleti logika - várólista és automatikus előreléptetés - működik.

A legfontosabb rövid távú feladat nem új feature, hanem a meglévő rendszer biztonsági és architekturális lezárása.

Ha az auth normalizálás és a permission layer elkészül, utána a projekt már nagyon jó alapot ad valódi felhasználói tesztekhez.